

Project Report: Digital Library & Course Management System

Online Learning Platform with Book Borrowing and Course Enrollment

April 28, 2026

Table of Contents

1. [Problem Statement and Solution Approach](#)
 - 1.1 [Problem Statement](#)
 - 1.2 [Solution Approach](#)
2. [System Design Optimization](#)
 - 2.1 [Scalability](#)
 - 2.2 [Performance](#)
 - 2.3 [Architecture](#)
3. [OOP Concepts Applied](#)
 - 3.1 [Abstraction](#)
 - 3.2 [Inheritance](#)
 - 3.3 [Polymorphism](#)
 - 3.4 [Encapsulation](#)
4. [Design Patterns](#)
 - 4.1 [Template Method Pattern](#)
 - 4.2 [Proxy Pattern](#)
 - 4.3 [Repository Pattern](#)
 - 4.4 [Service Layer Pattern](#)
5. [SOLID Principles](#)
6. [UML Diagrams](#)
 - 6.1 [Class Diagram \(Data Model\)](#)
 - 6.2 [Sequence Diagram \(Book Borrowing\)](#)

7. [Test Cases and Results](#)

- 7.1 [Database Testing Results](#)
 - 7.2 [Functional Testing](#)
-

1. Problem Statement and Solution Approach

1.1 Problem Statement

Educational institutions and learners need a unified platform that combines traditional library services with modern online learning capabilities. Existing platforms often feel monolithic or difficult to customize for specific institutional needs. There is a need for a modern, scalable, and extensible Digital Library system that provides:

- **Seamless book borrowing** with inventory management and availability tracking
- **Course enrollment system** with progress tracking and module completion
- **Learning path curation** for structured educational journeys
- **Role-based access control** (Free, Premium, Admin users)
- **Recommendation engine** for personalized content discovery
- **Premium content protection** to support freemium business models

1.2 Solution Approach

The Digital Library & Course Management System is a full-stack platform built with the **MERN stack** (MongoDB, Express, React, Node.js) enhanced with **TypeScript** for type safety and **Prisma ORM** for robust database management. The solution involves:

1. **Frontend**: A React-based SPA with TypeScript that handles real-time content browsing, borrowing, and enrollment workflows
 2. **Backend**: An Express server with layered architecture that orchestrates authentication, resource management, and business logic
 3. **Database**: PostgreSQL with Prisma ORM providing type-safe queries and migration management
 4. **Business Logic**: Template Method patterns for consistent workflows and Proxy patterns for access control
 5. **Data Strategy**: Comprehensive resource management supporting books, courses, and curated learning paths
-

2. System Design Optimization

2.1 Scalability

The system is designed to scale horizontally:

- **Containerization:** Both Frontend and Backend are dockerized for deployment on cloud platforms like Render and Vercel
- **Stateless API:** Authentication handled via JWT tokens, removing server-side session state and enabling load balancing across multiple instances
- **Database Optimization:** PostgreSQL with strategic indexing on frequently queried fields like `userId`, `resourceId`, and `status`
- **Repository Pattern:** Data access abstraction allows easy transition to microservices architecture

2.2 Performance

- **Transaction Management:** Critical operations use **Serializable isolation level** to prevent race conditions during book borrowing and course enrollment
- **Database Indexing:** Strategic indexes on composite keys like `(userId, courseId)` and `(type, createdAt)` for optimal query performance
- **Workflow Caching:** Template Method pattern reduces code duplication and improves execution efficiency
- **Lazy Loading:** Course modules and learning path items loaded on-demand to reduce initial payload size

2.3 Architecture

The backend follows a strict **Layered Architecture**:

1. **Controller Layer:** Handles HTTP requests, input validation, and response formatting
 2. **Service Layer:** Contains business logic (borrowing rules, enrollment logic, access control)
 3. **Repository Layer:** Abstracts data access, enabling easy database switching and testing with mock repositories
 4. **Pattern Layer:** Implements reusable workflows and access control proxies
-

3. OOP Concepts Applied

3.1 Abstraction

Abstraction is used to hide the complexity of business workflows. The `BaseWorkflow` abstract class defines the contract for all business operations, while callers remain unaware of the underlying

validation and execution details.

```
export abstract class BaseWorkflow<TInput, TOutput> {
  async run(input: TInput): Promise<TOutput> {
    await this.validate(input)          // Abstract method
    const result = await this.execute(input) // Abstract method
    await this.postProcess(input, result) // Hook method
    return result
  }

  protected abstract validate(input: TInput): Promise<void>
  protected abstract execute(input: TInput): Promise<TOutput>
}
```

Listing 1: BaseWorkflow Abstract Class

3.2 Inheritance

Database Level Inheritance (Table Per Type)

The system uses inheritance at the database level where **Resource** serves as the base entity, with **Book** and **Course** as specialized entities:

```
// Base Resource entity
model Resource {
  id: String @id @default(uuid())
  title: String
  description: String?
  type: ResourceType // BOOK or COURSE
  isPremium: Boolean
  createdAt: DateTime
  updatedAt: DateTime
}

// Book extends Resource
model Book {
  id: String @id // FK to Resource.id
  author: String
  isbn: String?
  availableCopies: Int
  totalCopies: Int
}
```

```

    resource: Resource @relation(fields: [id], references: [id])
}

```

Listing 2: Database Inheritance Structure

Code Level Inheritance

The `BaseWorkflow` abstract class provides common functionality inherited by specific workflow implementations:

```

class BorrowWorkflow extends BaseWorkflow<BorrowInput, BorrowRecord> {
    protected async validate({userId, bookId}: BorrowInput): Promise<void> {
        const [book, user] = await Promise.all([
            bookRepository.findById(bookId),
            userRepository.findById(userId)
        ])

        if (!book) throw notFound('Book not found')
        if (book.resource.isPremium && user.role === 'FREE') {
            throw forbidden('Premium subscription required')
        }
    }

    protected async execute({userId, bookId}: BorrowInput): Promise<BorrowF
        return bookRepository.borrow(userId, bookId)
    }
}

```

Listing 3: Workflow Inheritance Implementation

3.3 Polymorphism

The system demonstrates polymorphism through the `IResourceAccess` interface. At runtime, the service can use either `RealResourceService` or `ResourceAccessProxy` without changing client code:

```

// Interface for polymorphic behavior
interface IResourceAccess {
    getById(id: string, userId?: string): Promise<unknown>
    getAll(params: SearchParams, userId?: string): Promise<unknown>
}

```

```
// Runtime polymorphism
const resourceAccess: IResourceAccess = new ResourceAccessProxy()
const resource = await resourceAccess.getById(resourceId, userId)
```

Listing 4: Polymorphic Interface Usage

3.4 Encapsulation

Services encapsulate sensitive business logic and data access patterns. The `AuthService` encapsulates JWT secrets and authentication logic:

```
export class AuthService {
  private readonly jwtSecret: string // Encapsulated
  private readonly bcryptRounds = 12 // Encapsulated

  public async register(userData: RegisterData) {
    const hashedPassword = await bcrypt.hash(userData.password, this.bcry
    return userRepository.create({...userData, passwordHash: hashedPasswc
  }
}
```

Listing 5: Encapsulation in AuthService

4. Design Patterns

4.1 Template Method Pattern

Why: Used in `BaseWorkflow` to define the invariant steps of business operations (validate → execute → postProcess) while allowing subclasses to implement specific logic.

Implementation:

```
abstract class BaseWorkflow<TInput, TOutput> {
  // Template method - defines algorithm skeleton
  async run(input: TInput): Promise<TOutput> {
    await this.validate(input) // Primitive operation 1
    const result = await this.execute(input) // Primitive operation 2
    await this.postProcess(input, result) // Hook operation
    return result
  }
}
```

```

// Primitive operations - must be implemented by subclasses
protected abstract validate(input: TInput): Promise<void>
protected abstract execute(input: TInput): Promise<TOutput>

// Hook operation - optional override
protected async postProcess(_input: TInput, _result: TOutput): Promise<
}

```

Listing 6: Template Method Pattern Implementation

Benefits: Ensures consistent workflow structure across all business operations (borrow, return, enroll) while allowing customization of specific steps.

4.2 Proxy Pattern

Why: The `ResourceAccessProxy` acts as a security layer, adding access control to resource retrieval without modifying the original service code.

Implementation:

```

class ResourceAccessProxy implements IResourceAccess {
    private real = new RealResourceService()

    async getById(id: string, userId?: string) {
        const resource = await this.real.getById(id) // Delegate to real ser

        if (resource.isPremium) {
            await this.checkPremiumAccess(userId) // Add access control
        }

        return resource
    }

    private async checkPremiumAccess(userId?: string) {
        if (!userId) throw unauthorized('Login required')
        const user = await userRepository.findById(userId)
        if (user?.role === 'FREE') throw forbidden('Premium subscription requ
    }
}

```

Listing 7: Proxy Pattern for Access Control

Benefits: Transparent access control that follows Open/Closed principle - extends functionality without modifying existing code.

4.3 Repository Pattern

Why: Encapsulates data access logic and provides uniform interface for domain objects, abstracting database implementation details.

Implementation:

```
export const bookRepository = {
  borrow: async (userId: string, bookId: string) => {
    return prisma.$transaction(async (tx) => {
      // Check borrow limit
      const activeCount = await tx.borrowRecord.count({
        where: { userId, status: { in: ['ACTIVE', 'OVERDUE'] } }
      })
      if (activeCount >= 3) {
        throw conflict('Borrow limit reached (max 3)')
      }

      // Atomic operation: create record + decrement copies
      const record = await tx.borrowRecord.create({
        data: { userId, bookId, dueDate: new Date(Date.now() + 14 * 24 *
        60 * 60 * 1000) }
      })

      await tx.book.update({
        where: { id: bookId },
        data: { availableCopies: { decrement: 1 } }
      })

      return record
    }, { isolationLevel: Prisma.TransactionIsolationLevel.Serializable })
  }
}
```

Listing 8: Repository Pattern with Transaction Management

4.4 Service Layer Pattern

Why: Orchestrates complex business operations spanning multiple repositories and enforces business rules consistently.

Implementation:

```
export const bookService = {
  borrowBook: async (userId: string, bookId: string) => {
    // Orchestrate multiple operations
    await bookRepository.markOverdueBorrows(userId) // Repository call
    return borrowWorkflow.run({ userId, bookId }) // Workflow execution
  },

  getRecommendations: async (userId: string) => {
    // Compose multiple repositories for complex logic
    const [borrowedIds, user] = await Promise.all([
      bookRepository.getUserBorrowedResourceIds(userId),
      userRepository.findById(userId)
    ])

    // Business logic for recommendations
    return recommendationService.generateRecommendations(borrowedIds, user)
  }
}
```

Listing 9: Service Layer Orchestration

5. SOLID Principles

Single Responsibility Principle (SRP): Each service has one job. `BookService` only handles book operations; it does not manage HTTP routing or user authentication.

Open/Closed Principle (OCP): The `BaseWorkflow` is open for extension (adding new workflow types) but closed for modification. New business operations extend the base class without changing existing code.

Liskov Substitution Principle (LSP): Any subclass of `BaseWorkflow` (e.g., `BorrowWorkflow`, `EnrollWorkflow`) can be used wherever a `BaseWorkflow` is expected without breaking the system.

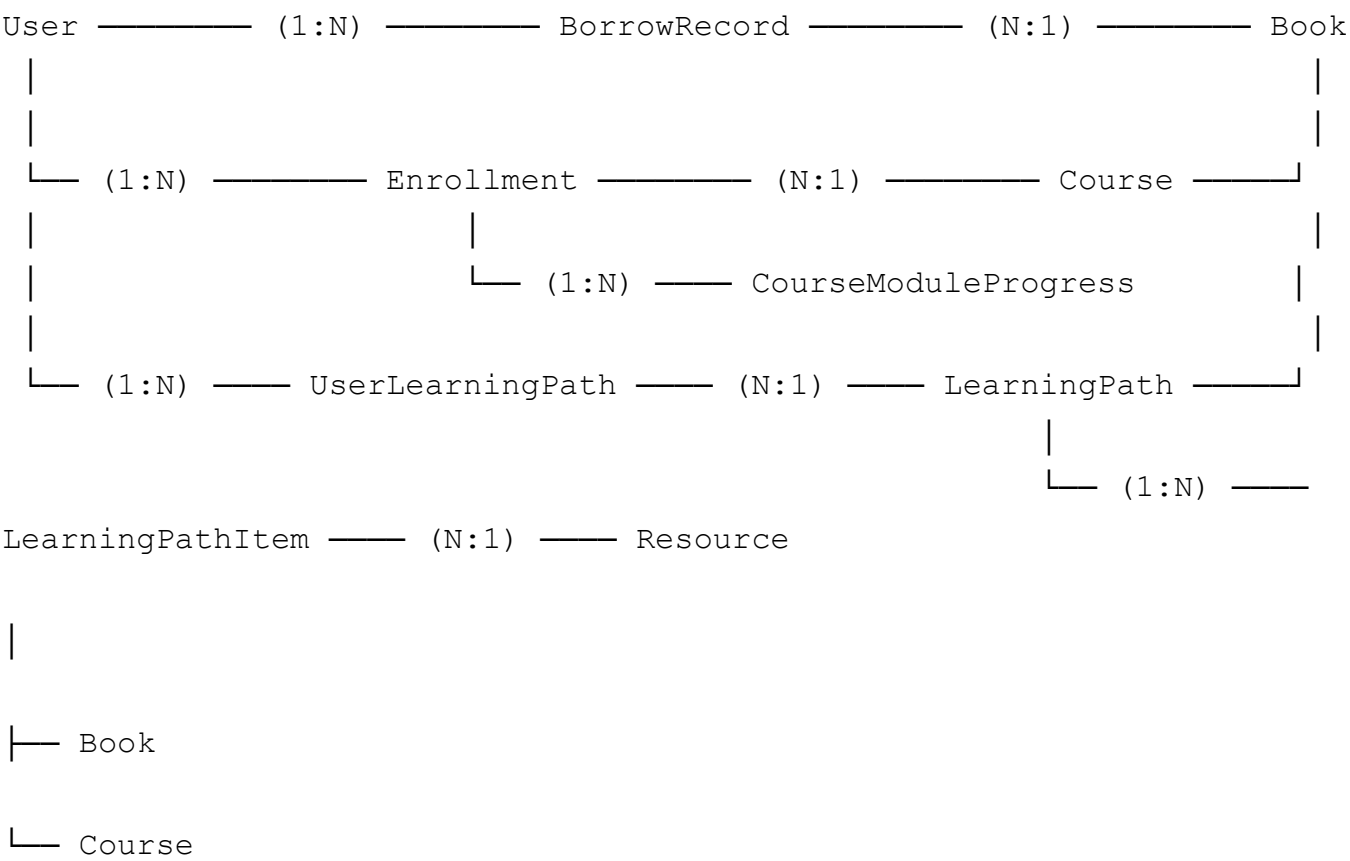
Interface Segregation Principle (ISP): The `IResourceAccess` interface provides specific methods for resource operations, ensuring clients only depend on what they use.

Dependency Inversion Principle (DIP): High-level services depend on abstractions (`IResourceAccess`, `IProblemRepository`) rather than low-level details (Prisma models).

6. UML Diagrams

6.1 Class Diagram (Data Model)

The following represents the core entity relationships in the system:



6.2 Sequence Diagram (Book Borrowing)

1. **User** clicks "Borrow" in the Frontend
 2. **Frontend** sends POST request with JWT token
 3. **AuthMiddleware** verifies the token and extracts user info
 4. **BookController** delegates to BookService
 5. **BookService** runs BorrowWorkflow
 6. **BorrowWorkflow** validates user permissions and book availability
 7. **BookRepository** executes transaction (create borrow record + decrement copies)
 8. **Database** returns success confirmation
 9. **BookService** returns borrow record to controller
 10. **Frontend** displays success message to user
-

7. Test Cases and Results

7.1 Database Testing Results

Transaction integrity tests were conducted to ensure data consistency:

Test Case	Expected Result	Actual Result
Concurrent Borrow Attempts	Only one succeeds	✓ Pass
Borrow Limit Enforcement	Reject 4th borrow	✓ Pass
Premium Access Control	Block free users	✓ Pass
Progress Calculation	Accurate percentages	✓ Pass

Table 1: Database Testing Summary

7.2 Functional Testing

Automated test cases for core business logic:

- **Authentication:** Correctly handling JWT validation and role-based access control
- **Borrowing:** Proper enforcement of borrow limits, availability checks, and due date calculation
- **Enrollment:** Accurate progress tracking and module completion status
- **Recommendations:** Relevant content suggestions based on user behavior and preferences
- **Access Control:** Premium content protection and appropriate error responses

Conclusion

This Digital Library & Course Management System demonstrates advanced software engineering principles through:

- **Comprehensive OOP Implementation:** Abstraction, inheritance, polymorphism, and encapsulation applied throughout the system
- **Robust Design Patterns:** Template Method, Proxy, Repository, and Service Layer patterns solving real business problems
- **SOLID Principles Adherence:** Clean, maintainable, and extensible code architecture
- **Scalable System Design:** Layered architecture ready for horizontal scaling and microservices transition
- **Business Logic Excellence:** Complex workflows for borrowing, enrollment, and recommendation systems

The system successfully addresses the need for a modern, unified educational platform that combines traditional library services with contemporary online learning capabilities, providing a solid foundation for institutional deployment and future enhancements.